Trabajo Práctico

Base de datos *key-value* replicado

Programación Concurrente y Paralela
Segundo cuatrimestre 2026

Entrega: 11 de noviembre de 2026 a las 23:59hs

1. Objetivo

Diseñar e implementar en Erlang un base de datos *key-value* replicado entre varios procesos de una misma máquina virtual. Se provee una API fija donde **el diseño interno corre por cuenta de cada grupo**. Eso significa: cómo se representa el estado, qué mensajes intercambian los procesos, cómo se resuelven los conflictos, cómo se administra la memoria, etc.

2. Bases de datos *key-value*

Una base de datos *key-value* es un sistema de almacenamiento en memoria en el que cada dato se guarda asociado a un identificador único, la **clave**, y solo se puede recuperar a través de ella. Desde el punto de vista del modelo de datos, es equivalente a un mapa o diccionario.

Clave	**Valor**
`usuario_42`	`#{nombre => "Ana", edad => 30}`
`sesion_9f3a`	`{logueado, usuario_42}`
`contador_visitas`	`1532`

A diferencia de las bases de datos relacionales, no hay esquema ni lenguaje de consultas. Las únicas operaciones son:

Operación	**Descripción**	**En este TP**
Escritura	Asocia un valor a una clave. Si la clave ya tenía un valor, lo reemplaza.	`put`
Lectura	Devuelve el valor asociado a una clave, o indica que no existe.	`get`
Borrado	Elimina la asociación de una clave.	`delete`

Gracias a esta simplicidad tienen bajo tiempo de respuesta y son fáciles de distribuir entre varios nodos. Se usan habitualmente como caché y para guardar sesiones, configuración o estado temporal.

En este trabajo, tanto las claves como los valores pueden ser cualquier término de Erlang.

3. Interfaz

El módulo tiene que exportar exactamente estas funciones, con estos tipos. **La API no se puede modificar.**

```
-module(simple_kv).
-export([start_link/1, stop/1, put/3, put/4, delete/2, get/2]).

-spec start_link(pos_integer()) -> {ok, [pid()]}.
-spec stop([pid()]) -> ok.
-spec put(pid(), term(), term()) -> ok.
-spec put(pid(), term(), term(), non_neg_integer() | infinity) -> ok.
-spec delete(pid(), term()) -> ok.
-spec get(pid(), term()) -> {ok, term()} | not_found.
```

`start_link(Count)` crea `Count` procesos enlazados al proceso que la invoca y devuelve sus identificadores. Cada uno de ellos es una réplica del almacén y puede atender cualquier operación. Si `Count` no es un entero positivo, la función falla con `function_clause`.

`put(Pid, Key, Value, TTL)` guarda o reemplaza el valor de `Key` en la réplica `Pid`, con el *time to live* indicado (ver sección 4.6). `put/3` equivale a `put/4` con `TTL = infinity`.

`delete(Pid, Key)` borra la clave. La operación es válida aunque la clave no exista.

`get(Pid, Key)` devuelve `{ok, Value}` si la clave tiene un valor vigente, o `not_found` en caso contrario.

`stop(Pids)` detiene todos los procesos de la lista y devuelve `ok` una vez que terminaron.

Ejemplo de uso

```
1> {ok, [P1, P2, P3]} = simple_kv:start_link(3).
2> simple_kv:put(P1, color, rojo).
ok
3> simple_kv:get(P2, color).
{ok,rojo}
4> simple_kv:delete(P1, color).
ok
5> simple_kv:get(P1, color).
not_found
6> simple_kv:put(P1, temp, 42, 50).
ok
7> timer:sleep(100), simple_kv:get(P1, temp).
not_found
8> simple_kv:put(P1, x, 1, -5).
** exception error: no function clause matching simple_kv:put(...)
9> simple_kv:stop([P1, P2, P3]).
ok
```

4. Replicación y consistencia

4.1. Réplicas

Una **réplica** es una copia completa e independiente de los datos. En este trabajo, cada proceso creado por `start_link/1` es una réplica: mantiene su propia copia de los datos y puede atender cualquier operación.

Tener varias réplicas permite repartir la carga de lectura y seguir operando aunque alguna no esté disponible. A cambio, aparece el problema central de este trabajo: **mantener la coherencia entre las copias**.

4.2. Replicación

La **replicación** es el mecanismo por el cual un cambio hecho en una réplica se propaga a las demás. Hay dos esquemas:

- **Sincrónico:** la operación termina recién cuando todas las réplicas confirmaron el cambio. Las copias siempre coinciden, pero la latencia la determina la réplica más lenta.
- **Asincrónico:** la operación se aplica en la réplica local y termina enseguida; las demás reciben el cambio después. La latencia es baja, pero las copias pueden diferir por un tiempo.

El sistema tiene que usar **replicación asincrónica**: todo cambio realizado en una réplica debe propagarse a todas las demás, y `put` y `delete` devuelven `ok` **sin esperar** a que las demás réplicas lo apliquen. Por lo tanto, la siguiente situación es válida:

```
t0   put(P1, color, rojo) -> ok     P1: rojo    P2: sin datos
t1   P2 recibe el cambio            P1: rojo    P2: rojo
```

Entre t_0 y t_1, las réplicas responden distinto.

4.3. Consistencia eventual

Como la replicación es asincrónica, no se puede exigir que las réplicas coincidan en todo momento. Se exige una propiedad más débil, la **consistencia eventual**: si dejan de producirse escrituras, todas las réplicas terminan con el mismo estado. A esto se le llama **convergencia**, y el sistema tiene que garantizarla.

4.4. Escrituras concurrentes y resolución de conflictos

Dos escrituras sobre la misma clave son **concurrentes** cuando se hacen en réplicas distintas sin que ninguna conozca la otra:

```
put(P1, color, rojo)            put(P2, color, azul)
```

Cada réplica puede recibir los dos cambios en un orden distinto. Si cada una conservara el último que recibió, podrían terminar con valores diferentes y no convergerían nunca.

Por eso hace falta una **política de resolución de conflictos**: un criterio que, dadas dos versiones de una misma clave, elija siempre la misma, **sin importar el orden en que llegaron**. La política que se use y su justificación son parte del diseño.

4.5. Borrado en un sistema replicado

Con una sola copia, borrar es simplemente quitar la clave. Con varias réplicas y mensajes que llegan con demora, eso no alcanza:

```
1. put(P1, color, rojo)     P1 envía el cambio a P3; el mensaje se demora.
2. delete(P2, color)        P2 envía el borrado a P3; llega primero.
3. P3 aplica el borrado     y elimina la clave.
4. P3 recibe el put         sin información sobre la clave, lo guarda.
```

El resultado es que P3 vuelve a tener un dato que había sido borrado. El sistema no puede permitir este comportamiento: **un dato borrado solo puede volver a aparecer mediante una nueva escritura**. El mecanismo para lograrlo es parte del diseño.

4.6. *Time to live* (TTL)

El **TTL** (*time to live*) es el tiempo durante el cual un valor es válido desde que se escribe. Cuando ese tiempo se cumple, el valor **vence** y el sistema tiene que comportarse como si no existiera: **nunca se devuelve un valor vencido**.

```
put(P1, codigo_verificacion, 482913, 60000).   %% válido durante 60 s
```

El TTL se expresa en milisegundos. `infinity` indica que el valor no vence y `0` que vence en el mismo momento en que se escribe. Cualquier otro valor que no sea un entero no negativo es inválido, y `put/4` tiene que fallar con `function_clause` en el proceso que la invoca. Cuándo y cómo se detecta el vencimiento queda a criterio de cada grupo.

4.7. Capacidad y desalojo

Cada réplica puede guardar como máximo **16 valores**. Cuando tiene que hacer lugar, **desaloja** algunos de los valores que guarda.

Desalojar no es lo mismo que borrar. Un borrado es un pedido explícito del usuario, a través de `delete`, y significa que el dato deja de existir en todo el sistema. Un desalojo, en cambio, es una decisión interna de una réplica por falta de espacio: esa réplica deja de guardar el dato, pero el dato sigue siendo válido.

La política de desalojo es parte del diseño. La única condición sobre qué valores conservar es que las **12 claves escritas más recientemente** sigan siempre disponibles. Qué otros criterios tener en cuenta para elegir los valores a desalojar queda a criterio de cada grupo, y tiene que estar justificado en el documento de diseño. Además, como un valor desalojado sigue siendo válido, antes de descartar un valor vigente la réplica tiene que dar a las demás la oportunidad de conservarlo. Este intercambio no puede desencadenar una secuencia de mensajes entre réplicas que no termine.

4.8. Lectura con consulta a otras réplicas

Una réplica puede no tener información sobre una clave que sí existe en el sistema, por ejemplo porque la desalojó o porque todavía no recibió una escritura hecha en otra réplica. En ese caso, responder `not_found` sería incorrecto. Por eso, cuando la réplica consultada no tiene información sobre una clave, **la respuesta tiene que tener en cuenta la información de las demás réplicas**.

5. Aspectos de concurrencia

5.1. Comunicación sincrónica sobre mensajes asincrónicos

En Erlang, el envío de mensajes (`!`) es asincrónico: el emisor no espera ninguna confirmación. Sin embargo, para quien usa la API, las operaciones son sincrónicas: cada llamada devuelve un resultado.

Implementar un protocolo de pedido y respuesta sobre mensajes asincrónicos, que asocie cada respuesta con el pedido que la originó, es parte del trabajo. Esto es justamente lo que resuelve `gen_server`, cuyo uso no está permitido. Los procesos tienen que descartar los mensajes que no esperan, de modo que no se acumulen en su buzón.

5.2. Deadlock

El diseño tiene que garantizar que no pueda producirse un *deadlock* entre réplicas. En particular, una réplica que espera información de las demás tiene que poder seguir atendiendo otros pedidos mientras tanto.

5.3. Orden de entrega de mensajes

Erlang garantiza que, si un proceso A le envía dos mensajes a un proceso B, B los recibe en el mismo orden en que se enviaron.

No hay ninguna garantía entre emisores distintos: si A y B le envían un mensaje cada uno a C, C los puede recibir en cualquier orden, más allá de cuál se haya enviado primero. Hay que tener esto en cuenta tanto en el diseño como en los tests.

6. Restricciones

No se puede usar:

- `gen_server`, `gen_statem`, `gen_event`, `supervisor` ni ningún otro *behaviour* de OTP.
- `ets`, `dets`, `mnesia`, `persistent_term`.
- El diccionario del proceso (`erlang:put/2`, `erlang:get/1`).

El estado de cada proceso tiene que vivir únicamente en los argumentos de su ciclo de recepción de mensajes. El resto de la biblioteca estándar está permitido. El código tiene que compilar sin advertencias con `erlc +warn_all`.

7. Supuestos

- Todos los procesos se ejecutan en una única máquina virtual BEAM.
- Los procesos no fallan.
- `stop/1` se invoca una única vez, al final, con todos los identificadores devueltos por `start_link/1`.

8. Entrega

1. Los archivos `.erl` de la implementación.
2. Cada grupo tiene que crear sus propios tests, que muestren el cumplimiento de cada uno de los comportamientos pedidos en este enunciado.
3. **Documento de diseño** (máximo 5 páginas), que incluya:
   - **Estructura de las réplicas**: qué información guardan y por qué.
   - **Protocolo de comunicación**: una tabla con todos los mensajes, indicando emisor, receptor, contenido y efecto en quien lo recibe.
   - **Estrategia para resolución de conflictos y borrados**: mecanismo elegido y justificación de su corrección.
   - **Mecanismo de detección de vencimiento**: cuándo y cómo se detecta que un valor expiró.
   - **Mecanismo de desalojo**: política de desalojo adoptada y justificación de que no genera secuencias de mensajes que no terminan.
   - **Algortimo para `get`**: funcionamiento de un `get` que requiera información de otras réplicas, y por qué no produce un *deadlock*.

Recordar que los tests prueban únicamente el comportamiento observable a través de la interfaz externa. Que una implementación los pase no asegura que su diseño sea correcto: la corrección del diseño se evalúa a partir del código y del documento de diseño.